

Minería de datos y Modelización predictiva

Análisis de la abstención en las elecciones de los
municipios de España

Adolfo Viguera Varea
Máster Big Data, Data Science, Inteligencia Artificial

1. Introducción al problema y variables implicadas

Se han elegido como variables objetivo ambas variables relacionadas con la abstención:

- Variable objetivo continua (regresión lineal): AbstentionPtge
- Variable objetivo binaria (regresión logística): AbstencionAlta

Por lo tanto, el objetivo del problema es crear dos modelos:

- Un modelo de regresión lineal que permita predecir el porcentaje de abstención que hay en un municipio de España en base a datos demográficos de la región.
- Un modelo de regresión logística que permita determinar si hay una alta abstención o no en un municipio de España.

El resto de variables objetivo serán eliminadas.

Respecto a los datos, cada observación representa un municipio de España. A partir de las variables, descubrimos que provincia y CCAA pertenece, el porcentaje de mujeres, el porcentaje de extranjeros, el porcentaje de personas en rangos de edad, el número de empresas por sector y muchos más datos relacionados con el municipio.

2. Importación del conjunto de datos

Importamos el conjunto de datos y eliminamos las variables objetivo que hemos descartado

```
datos = pd.read_excel("datosEleccionesEspaña.xlsx")

datos.drop(['Izda_Pct', 'Dcha_Pct', 'Otros_Pct', 'Izquierda', 'Derecha'],
           axis=1, inplace=True)
```

3. Análisis descriptivo del conjunto de datos

```
datos.info()
```

En este conjunto de datos, encontramos un total de 36 variables. A priori 32 son numéricas y 4 categóricas.

Vamos a comprobar si hay alguna numérica que realmente sea categórica

```
cuentaDistintos(datos)
```

Tanto la variable explicativa CodigoProvincia como la variable objetivo AbstencionAlta son variables numéricas que realmente representan categorías. No representan un orden numérico continuo, sino una clasificación en grupos. Por ello, hay que transformarlas a categóricas

```
for var in ["CodigoProvincia", "AbstencionAlta"]:
```

```

datos[var] = datos[var].astype(str)

# Seleccionar las columnas numéricas del DataFrame
numericas = datos.select_dtypes(include=['int', 'int32', 'int64', 'float', 'float32', 'float64']).columns

# Seleccionar las columnas categóricas del DataFrame
categoricas = [variable for variable in datos.columns if variable not in numericas]

```

Ahora, revisamos ambos tipos de variables para encontrar errores y valores extraños

Primero, las categóricas.

```
analizar_variables_categoricas(datos)
```

Se encontraron los siguientes errores:

- En la variable densidad, hay una categoría llamada "?"
- En la variable Name, esperaríamos encontrar un nombre para cada municipio y así utilizar esta variable como ID. Pero, resulta que existen dos municipios con el mismo nombre pero de comunidades diferentes. Por lo tanto, tendremos que modificar el nombre para poder utilizarlo como ID

Ahora, analizamos las variables numéricas:

```

descripcion_num = datos.describe()
for num in numericas:
    descripcion_num.loc["Asimetria", num] = datos[num].skew()
    descripcion_num.loc["Kurtosis", num] = datos[num].kurtosis()
    descripcion_num.loc["Rango", num] = datos[num].max() - datos[num].min()

# Configurar opciones de visualización de pandas
descripcion_num = descripcion_num.round(3)

descripcion_num.T

```

- Age_over65_pct: tiene un valor mínimo negativo (-18.052).
- ForeignersPtge: lo mismo, un valor mínimo negativo (-8.960)

Observación: No podemos transformar todos los valores negativos de porcentajes en np.nan ya que la variable PobChange_pct tiene porcentajes negativos que indican la disminución de población respecto a las anteriores elecciones

- SameComAutonPtge: valor máximo de 127.156. No puede ser ya que es un porcentaje
- Explotaciones: Valor máximo extraño de número de explotaciones agrícolas: 99999.000.

4. Corrección de errores

Transformamos la categoría ? de densidad en NaN y convertimos los nombres en valores únicos añadiendo la CCAA a la que pertenecen. Luego, transformamos la columna Name a índice

```
datos['Densidad'] = datos['Densidad'].replace('?', np.nan)

# Identificar los nombres duplicados
nombres_duplicados = datos[datos.duplicated(subset=['Name'], keep=False)]

for nombre in nombres_duplicados['Name'].unique():
    mascara = datos['Name'] == nombre
    datos.loc[mascara, 'Name'] = datos.loc[mascara, 'Name'] + '_' + datos.loc[mascara, 'CCAA']
```

```
Número de valores únicos en Name: 8117
Contador de valores de la variable densidad: Densidad
MuyBaja    6416
Baja       1053
Alta        556
Name: count, dtype: int64
```

```
datos = datos.set_index('Name')
```

Respecto a las variables numéricas, cambiamos los valores fuera de rango y valores extraños como 99999 en NaN.

```
# Lista de variables a corregir
variables_porcentaje = ['Age_19_65_pct', 'Age_over65_pct', 'ForeignersPtge', "SameComAutonPtge"]

# Transformar valores negativos en np.nan y valores > 100 en np.nan
for var in variables_porcentaje:
    # Valores negativos a np.nan
    datos.loc[datos[var] < 0, var] = np.nan
    # Valores mayores a 100 a np.nan (son porcentajes)
    datos.loc[datos[var] > 100, var] = np.nan

datos['Explotaciones'] = datos['Explotaciones'].replace(99999, np.nan)
```

5. Análisis de valores atípicos

En primer lugar, se analizaron el porcentaje de valores atípicos.

Debido a estos porcentajes y al significado de cada variable, se decidió lo siguiente:

- Mantener los valores atípicos de población y de censo ya que es normal que haya poblaciones con mayor o menor tamaño, sobre todo si comparamos un municipio pequeño con ciudades como Valencia o Barcelona
- Los demás valores atípicos pertenecen a porcentajes. Se ha decidido sustituirlos por valores missing.

```

resultados = {x: atipicosAmissing(datos_input[x])[1] / len(datos_input) for x in numericas_input}
resultados
✓ 0.2s
{'CodigoProvincia': 0.0,
 'Population': 0.0990513736602193,
 'TotalCensus': 0.09634101268941728,
 'Age_0-4_Ptge': 0.0,
 'Age_under19_Ptge': 0.0,
 'Age_19_65_pct': 0.002833559196747567,
 'Age_over65_pct': 0.0,
 'WomanPopulationPtge': 0.002587162744856474,
 'ForeignersPtge': 0.0,
 'SameComAutonPtge': 0.0,
 'SameComAutonDiffProvPtge': 0.020327707281015153,
 'DifComAutonPtge': 0.004927929037821855,
 'UnemployLess25_Ptge': 0.003203153874584206,
 'Unemploy25_40_Ptge': 0.0,
 'UnemployMore40_Ptge': 0.0,
 'AgricultureUnemploymentPtge': 0.019958112603178514,
 'IndustryUnemploymentPtge': 0.005913514845386226,
 'ConstructionUnemploymentPtge': 0.006529505975113958,
 'ServicesUnemploymentPtge': 0.0,
 'totalEmpresas': 0.0,
 'Industria': 0.0,
 'Construccion': 0.0,
 'ComercTTEHosteleria': 0.0,
 'Servicios': 0.0,
 'inmuebles': 0.0,
 'Pob2010': 0.0,
 'SUPERFICIE': 0.0,
 'PobChange_pct': 0.0,
 'PersonasInmueble': 0.0,
 'Explotaciones': 0.0}

```

```

# Modifico los atipicos como missings.

variables_a_modificar = [var for var in numericas_input if var not in ['Population', 'TotalCensus']]

for variable_numérica in variables_a_modificar:
    datos_input[variable_numérica] = atipicosAmissing(datos_input[variable_numérica])[0]

```

6. Análisis de valores perdidos. Imputaciones

A nivel de columna, analizamos la cantidad de NaNs y su proporción. No hay ninguna variable con más de la mitad de datos perdidos, por lo que no eliminamos ninguna.

```

prop_missingsVars = datos_input.isna().sum()/len(datos_input)
prop_missingsVars

```

A nivel de fila, calculamos la columna prop_missings y la añadimos a los datos de entrada. Esta columna representa información extra sobre el porcentaje de missings por observación. Si alguno de estos superase el 50%, eliminaríamos la observación pero no es el caso

```

# Creamos la variable prop_missings que recoge la proporción de valores perdidos por cada observación
datos_input['prop_missings'] = datos_input.isna().mean(axis=1)

# Realizamos un estudio descriptivo básico de la nueva variable
print(datos_input['prop_missings'].describe())

```

Además, solo hay 8 valores distintos en prop_missings, por lo que la transformamos en categórica.

```
datos_input["prop_missings"] = datos_input["prop_missings"].astype(str)

variables_input.append('prop_missings')
categoricas_input.append('prop_missings')
```

Respecto a las imputaciones de valores, en este conjunto de datos contamos con varios grupos de variables que son complementarias. Es decir, conocemos el valor de la suma de los valores de ciertas variables. Por ejemplo, sabemos que los grupos de variables que representan porcentajes (como la edad) deben sumar 100. Lo mismo ocurre con las variables relacionadas con el número de empresas, el total debe sumar lo que indica la variable totalEmpresas.

Si solo falta un valor en ese grupo de variables (por observación), se puede calcular restándolo de 100.

```
# Definición de grupos
grupo1 = ['Age_under19_Ptge', 'Age_19_65_pct', 'Age_over65_pct']
grupo2 = ['SameComAutonPtge', 'SameComAutonDiffProvPtge', 'DifComAutonPtge']
grupo3 = ['UnemployLess25_Ptge', 'Unemploy25_40_Ptge', 'UnemployMore40_Ptge']

# Función para imprimir filas con más de un NaN en un grupo
def check_missing_in_group(df, group, group_name):

    missing_count = df[group].isnull().sum(axis=1)
    # Selecciona las filas que tienen más de un NaN
    filas_con_muchos_nan = df[missing_count > 1]
    if not filas_con_muchos_nan.empty:
        print(f"En {group_name} se encontraron {len(filas_con_muchos_nan)} filas con más de un valor faltante:")
        print(filas_con_muchos_nan[group])
    else:
        print(f"No se encontraron filas con más de un valor faltante en {group_name}.")

# Aplicar la función a cada grupo
check_missing_in_group(datos, grupo1, "Porcentajes de edad")
check_missing_in_group(datos, grupo2, "Porcentajes de residencia")
check_missing_in_group(datos, grupo3, "Porcentajes de desempleo por edad")
```

```
En Porcentajes de edad se encontraron 1 filas con más de un valor faltante:
      Age_under19_Ptge  Age_19_65_pct  Age_over65_pct
Name
Illán de Vacas        0.0          NaN          NaN
No se encontraron filas con más de un valor faltante en Porcentajes de residencia.
No se encontraron filas con más de un valor faltante en Porcentajes de desempleo por edad.
```

En el caso de los grupos estipulados como grupo1, grupo2 y grupo3, solo hay una fila con más de un valor faltante. El resto, pueden imputarse restando el total a 100 de forma sencilla

```
# Función para rellenar el valor faltante en un grupo de variables complementarias
def fill_complementary_group(row, columns):

    missing_count = row[columns].isnull().sum()
    if missing_count == 1:

        sum_present = row[columns].sum(skipna=True)
        missing_value = 100 - sum_present
        missing_column = row[columns].isnull().idxmax()
        row[missing_column] = missing_value

    return row

# Definición de los grupos complementarios:

# Grupo 1: Porcentajes de edad
grupo1 = ['Age_under19_Ptge', 'Age_19_65_pct', 'Age_over65_pct']

# Grupo 2: Porcentajes de residencia
grupo2 = ['SameComAutonPtge', 'SameComAutonDifProvPtge', 'DifComAutonPtge']

# Grupo 3: Porcentajes de desempleo por edad
grupo3 = ['UnemployLess25_Ptge', 'Unemploy25_40_Ptge', 'UnemployMore40_Ptge']

# Aplicamos la función a cada fila del DataFrame para cada grupo
for grupo in [grupo1, grupo2, grupo3]:
    datos = datos.apply(lambda row: fill_complementary_group(row, grupo), axis=1)
```

Sin embargo, en el caso de los grupos 4 y 5, parece que falta una columna para cubrir todos los porcentajes (grupo4) o el total de empresas (grupo5). Por ejemplo, para el grupo4, en muchas observaciones el total no suma 100, ya que parece que falta otra columna que representa el resto de porcentaje de desempleados en otros sectores.

```
grupo4 = ['AgricultureUnemploymentPtge', 'IndustryUnemploymentPtge',
          'ConstructionUnemploymentPtge', 'ServicesUnemploymentPtge']

grupo5 = ['totalEmpresas', 'Industria', 'Construccion', 'ComercTTEHosteleria',
          'Servicios']

datos[grupo5].sample(5)
```

	totalEmpresas	Industria	Construccion	ComercTTEHosteleria	Servicios
Name					
Roda de Eresma	17.0	0.0	0.0	0.0	0.0
Mahora	80.0	11.0	12.0	40.0	17.0
Cascante del Río	0.0	0.0	0.0	0.0	0.0
Cazalegas	141.0	9.0	26.0	69.0	37.0
Hinojos	140.0	12.0	12.0	78.0	38.0

La lógica nos dice que "existe" una columna faltante que representa esos datos complementarios. Por lo tanto, si no hay más NaN en una observación, podríamos calcular esa columna. Sin embargo, como no podemos confirmar con el conjunto de datos que esa columna sea real, en este caso se procede con las imputaciones vistas en clase.

```
## IMPUTACIONES
# Imputo todas las cuantitativas, seleccionar el tipo de imputacion: media, mediana o aleatorio
for x in numericas_input:
    datos_input[x] = ImputacionCuant(datos_input[x], 'aleatorio')

# Imputo todas las cualitativas, seleccionar el tipo de imputacion: moda o aleatorio
for x in categoricas_input:
    datos_input[x] = ImputacionCuali(datos_input[x], 'aleatorio')
```

Tras comprobar que no quedaban NaNs y que los valores de cada variable tenían sentido y estaban dentro de sus rangos, se almacenaron los datos para su posterior uso mediante el siguiente código.

```
datosEleccionesLimpios = pd.concat([varObjBin, varObjCont, datos_input], axis = 1)
with open('datosEleccionesDep.pickle', 'wb') as archivo:
    pickle.dump(datosEleccionesLimpios, archivo)
```

7. Detección de relaciones

7.1. Detección de relaciones entre variables input continuas

Creamos un mapa de calor con las relaciones entre variables.

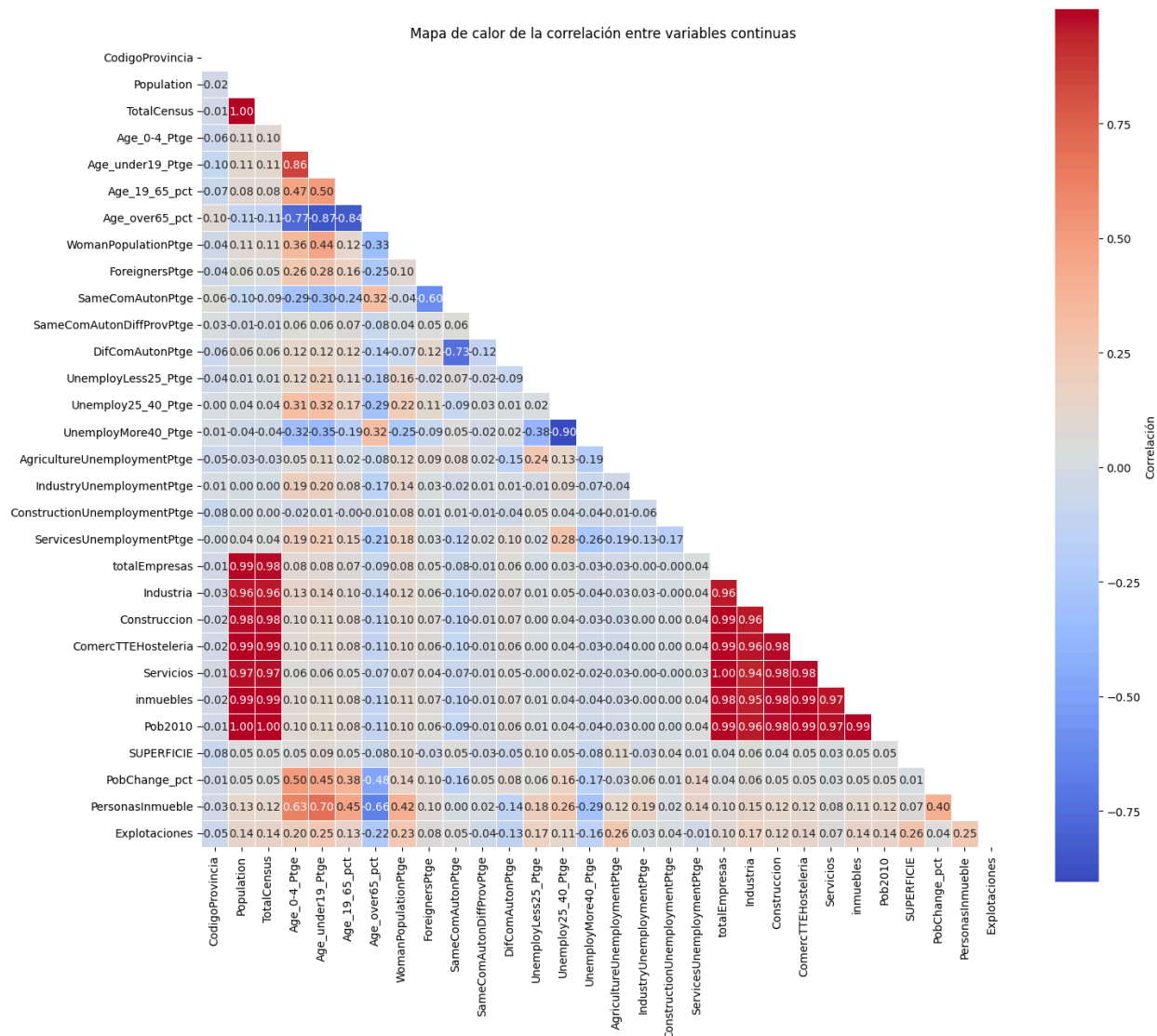
```
# Calcular la matriz de correlación
matriz_correlacion = datos_input[numericas_input].corr()

# Crear una máscara para la mitad superior
mask = np.triu(np.ones_like(matriz_correlacion, dtype=bool))

# Configurar el tamaño de la figura
plt.figure(figsize=(16, 14))

# Dibujar el mapa de calor con la máscara
sns.heatmap(matriz_correlacion, mask=mask, annot=True, fmt=".2f", cmap='coolwarm',
            cbar_kws={'label': 'Correlación'}, square=True, linewidths=0.5, annot_kws={"size": 10})

plt.title('Mapa de calor de la correlación entre variables continuas')
plt.show()
```



Observamos como hay una relación totalmente lineal entre variables como Population, TotalCensus y Pob2010. Además, relaciones casi de 1 (>0.95) entre variables de población y totalcensus con el número de empresas por sector y por lo tanto el total de empresas. Esto quiere decir que:

- A mayor población, el censo es mayor lo cual es lógico
- A mayor población, mayor número de empresas.

Observamos otras relaciones curiosas:

- El número de personas por inmueble está relacionado con el porcentaje de personas por rango de edad. Es decir, si hay mayor número de personas por debajo de los 19 años, el número de personas por inmueble aumenta, lo cual tiene sentido ya que en esta edad las personas suelen vivir en casa de sus padres y por lo tanto un único inmueble es ocupado por varias personas. Al contrario, si el porcentaje de personas por encima de los 65 años aumenta, el número de personas por inmueble disminuye, lo cual puede estar relacionado con que personas mayores opten por residencias en lugar de inmuebles propios.

- También hay una relación muy fuerte entre el número de empresas de distintos sectores. Es decir, parece ser que las empresas se encuentran en municipios específicos (grandes ciudades) donde no sólo se encuentran de un tipo sino de muchos.

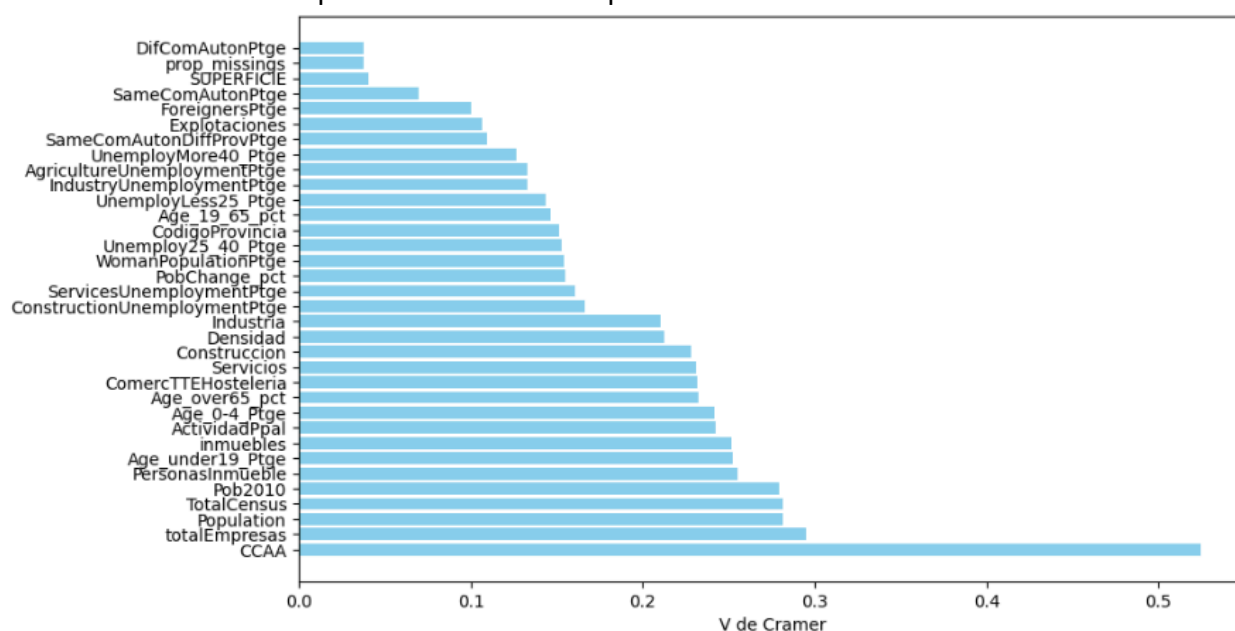
Estas relaciones entre variables generan bastante colinealidad, como se puede observar después en la parte de las regresiones.

7.2. Relaciones entre variables input y objetivo

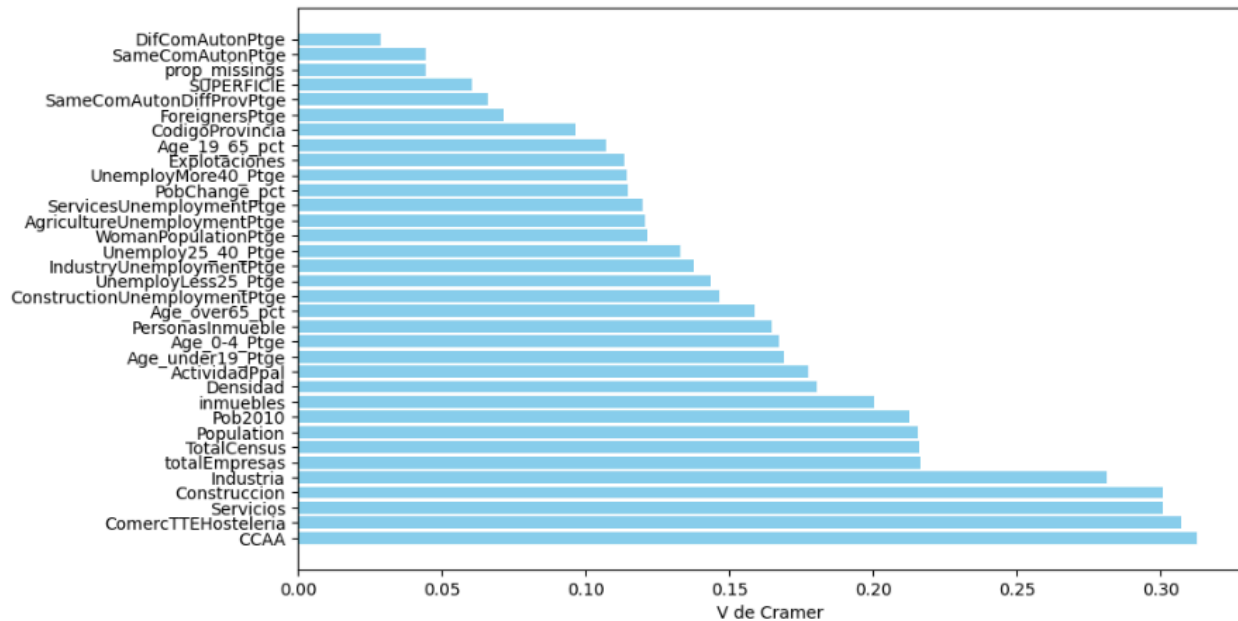
A continuación, utilizamos el estadístico V de Cramer para medir la relación entre las variables predictoras y las objetivo. Debido a como están creadas las funciones VCramer y graficoVcramer, podemos comparar cualquier variable predictora con las objetivo ya que las numéricas son discretizadas y convertidas a categóricas antes de calcular el estadístico.

Idealmente se debería calcular la correlación para comparar variables numéricas, pero esta opción también nos permite visualizar y obtener una relación fiable entre variables.

Valores de V de cramer para cada variable respecto a la variable binaria:



Valores de V de Cramer respecto a la variable objetivo continua:



Como podemos observar en ambas gráficas, la relación entre la comunidad autónoma o el código de provincia con las variables objetivo es considerable. Sobre todo, para la target binaria AbstencionAlta.

Es decir, el lugar desde donde se vota influye mucho en la decisión de abstenerse o no a ello.

Por otro lado, variables como la proporción de missings o la superficie del municipio no influyen casi nada en las variables objetivo. Respecto a la superficie, tiene bastante sentido ya que siguiendo la lógica, no tiene un sentido razonable para influir en las votaciones. En el caso de la proporción de valores faltantes, parece ser que tampoco influyen.

8. Regresión lineal

Para descubrir cuál es el mejor modelo de regresión posible partiendo del conjunto de variables que tenemos, se van a aplicar los métodos de selección de variables

Antes de aplicarlos, dividimos el conjunto de datos en train y test.

```
# Obtengo la partición. np.ravel es para convertirlo en un array 1D
x_train, x_test, y_train, y_test = train_test_split(datos_input, np.ravel(varObjCont),
                                                    test_size = 0.2, random_state = 123456)
```

8.1. Selección clásica de variables

Para todas las variables originales: Métodos backward, forward y stepwise, todos utilizando las métricas AIC y BIC.

```

modeloStepAIC = lm_stepwise(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloBackAIC = lm_backward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloFrontAIC = lm_forward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloStepBIC = lm_stepwise(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloBackBIC = lm_backward(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloFrontBIC = lm_forward(y_train, x_train, var_cont, var_categ, [], 'BIC')

```

A continuación, se probó el método stepwise pero añadiendo todas las interacciones entre variables continuas al conjunto de datos. Lo hacemos para AIC y BIC,.

```

#Combinación de todas las variables continuas 2 a 2
interacciones_varcont = list(itertools.combinations(var_cont, 2))

```

```

# Selección de variables Stepwise, métrica AIC, con interacciones
modeloStepAIC_int = lm_stepwise(y_train, x_train, var_cont, var_categ, interacciones_varcont, 'AIC')
# Selección de variables Stepwise, métrica BIC, con interacciones
modeloStepBIC_int = lm_stepwise(y_train, x_train, var_cont, var_categ, interacciones_varcont, 'BIC')

```

A continuación, se ejecutó el siguiente código para obtener una tabla comparativa de los modelos en base a sus valores de R2 para entrenamiento, R2 para test y número de parámetros.

```

# Create a comparison table
models = {
    'Stepwise AIC': {'model': modeloStepAIC, 'x_test': x_test_modeloStepAIC},
    'Backward AIC': {'model': modeloBackAIC, 'x_test': x_test_modeloBackAIC},
    'Stepwise BIC': {'model': modeloStepBIC, 'x_test': x_test_modeloStepBIC},
    'Backward BIC': {'model': modeloBackBIC, 'x_test': x_test_modeloBackBIC},
    'Forward AIC': {'model': modeloFrontAIC, 'x_test': x_test_modeloFrontAIC},
    'Forward BIC': {'model': modeloFrontBIC, 'x_test': x_test_modeloFrontBIC},
    'Stepwise AIC con interacciones': {'model': modeloStepAIC_int, 'x_test': x_test_modeloStepAIC_int},
    'Stepwise BIC con interacciones': {'model': modeloStepBIC_int, 'x_test': x_test_modeloStepBIC_int}
}

# Create DataFrame
comparison = pd.DataFrame(columns=['R2 Train', 'R2 Test', 'Number of Parameters'])

for name, model_dict in models.items():
    model = model_dict['model']
    x_test_model = model_dict['x_test']

    r2_train = Rsq(model['Modelo'], y_train, model['X'])
    r2_test = Rsq(model['Modelo'], y_test, x_test_model)
    n_params = len(model['Modelo'].params)

    comparison.loc[name] = [r2_train, r2_test, n_params]

# Format floating point numbers
comparison[['R2 Train', 'R2 Test']] = comparison[['R2 Train', 'R2 Test']].round(4)

print(comparison)

```

	R2 Train	R2 Test	Number of Parameters
Stepwise AIC	0.4286	0.4273	72.0
Backward AIC	0.4309	0.4294	76.0
Stepwise BIC	0.4244	0.4212	63.0
Backward BIC	0.4280	0.4236	67.0
Forward AIC	0.4286	0.4274	73.0
Forward BIC	0.4244	0.4212	63.0
Stepwise AIC con interacciones	0.4629	0.4221	111.0
Stepwise BIC con interacciones	0.4444	0.4294	71.0

Comparando el valor de R2 y el número de parámetros y siguiendo el principio de parsimonia, los mejores modelos sin tener en cuenta interacciones son Stepwise BIC y ForwardBIC, ya que tienen un R2 similar y el menor número de parámetros, siendo entonces los modelos simples. Respecto a los modelos con interacciones, hay una clara diferencia en el número de parámetros, por lo que nos quedamos con el Stepwise BIC con interacciones.

Entonces, nos quedamos con:

- Modelo StepwiseBIC (es el mismo que ForwardAIC)
- Modelo StepwiseBIC con interacciones

8.2. Selección aleatoria de variables

Para aplicar el método de selección aleatoria de variables, se hizo uso del siguiente código, donde los modelos de regresión fueron entrenados utilizando el método de selección stepwise usando el criterio BIC. Además, se incluyeron todas las interacciones entre variables continuas, por lo que el tiempo de ejecución fue bastante largo (431 minutos)

```

# Inicializar un diccionario para almacenar las fórmulas y variables seleccionadas.
variables_seleccionadas = {
    'Formula': [],
    'Variables': []
}

# Realizar 30 iteraciones de selección aleatoria.
for x in range(30):
    print('----- iter: ' + str(x))

    # Dividir los datos de entrenamiento en conjuntos de entrenamiento y prueba.
    x_train2, x_test2, y_train2, y_test2 = train_test_split(x_train, y_train,
                                                            test_size = 0.3, random_state = 1234567 + x)

    # Realizar la selección stepwise utilizando el criterio BIC en la submuestra.
    modelo = lm_stepwise(y_train2.astype(int), x_train2, var_cont, var_categ, interacciones_varcont, 'BIC')

    # Almacenar las variables seleccionadas y la fórmula correspondiente.
    variables_seleccionadas['Variables'].append(modelo['Variables'])
    variables_seleccionadas['Formula'].append(sorted(modelo['Modelo'].model.exog_names))

# Unir las variables en las fórmulas seleccionadas en una sola cadena.
variables_seleccionadas['Formula'] = list(map(lambda x: '+'.join(x), variables_seleccionadas['Formula']))

# Calcular la frecuencia de cada fórmula y ordenarlas por frecuencia.
frecuencias = Counter(variables_seleccionadas['Formula'])
frec_ordenada = pd.DataFrame(list(frecuencias.items()), columns = ['Formula', 'Frecuencia'])
frec_ordenada = frec_ordenada.sort_values('Frecuencia', ascending = False).reset_index()

# Identificar las tres fórmulas más frecuentes y las variables correspondientes.
var_1 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][0])]
var_2 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][1])]
var_3 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][2])]

```

Como resultado, se obtuvieron 3 modelos candidatos a ganador. Estos 3 modelos son los que presentan las 3 fórmulas más frecuentes

Variables de la fórmula 1: {'cont': ['Age_over65_pct', 'WomanPopulationPtge'], 'categ': ['CodigoProvincia'], 'inter': [('UnemployMore40_Ptge', 'Explotaciones'), ('ConstructionUnemploymentPtge', 'ServicesUnemploymentPtge'), ('Age_under19_Ptge', 'Age_19_65_pct'), ('Age_under19_Ptge', 'SameComAutonPtge'), ('UnemployLess25_Ptge', 'AgricultureUnemploymentPtge'), ('Age_over65_pct', 'IndustryUnemploymentPtge'), ('SameComAutonPtge', 'ServicesUnemploymentPtge'), ('Age_over65_pct', 'SameComAutonDiffProvPtge'), ('ForeignersPtge', 'PobChange_pct'), ('DifComAutonPtge', 'SUPERFICIE'), ('UnemployLess25_Ptge', 'PobChange_pct')]]}

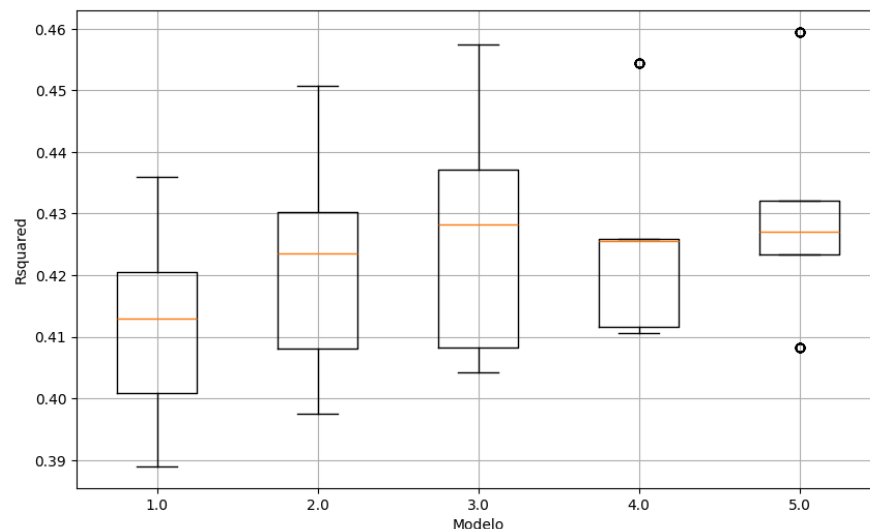
Variables de la fórmula 2: {'cont': [], 'categ': ['CodigoProvincia', 'ActividadPpal'], 'inter': [('UnemployMore40_Ptge', 'Explotaciones'), ('Age_19_65_pct', 'SameComAutonPtge'), ('Age_over65_pct', 'DifComAutonPtge'), ('Age_over65_pct', 'ServicesUnemploymentPtge'), ('DifComAutonPtge', 'ConstructionUnemploymentPtge'), ('Age_over65_pct', 'IndustryUnemploymentPtge'), ('Age_under19_Ptge', 'Age_19_65_pct'), ('Age_under19_Ptge',

'WomanPopulationPtge'), ('ForeignersPtge', 'IndustryUnemploymentPtge'), ('Age_over65_pct', 'SUPERFICIE'), ('TotalCensus', 'AgricultureUnemploymentPtge'), ('ComercTTEHosteleria', 'Explotaciones'), ('Age_over65_pct', 'SameComAutonDiffProvPtge'), ('ForeignersPtge', 'DifComAutonPtge'))]]

Variables de la fórmula 3: {'cont': ['Age_under19_Ptge'], 'categ': ['CodigoProvincia', 'ActividadPpal'], 'inter': [('Age_under19_Ptge', 'Age_19_65_pct'), ('Age_19_65_pct', 'SameComAutonPtge'), ('UnemployMore40_Ptge', 'Explotaciones'), ('IndustryUnemploymentPtge', 'ConstructionUnemploymentPtge'), ('Age_19_65_pct', 'Age_over65_pct'), ('ConstructionUnemploymentPtge', 'ServicesUnemploymentPtge'), ('Age_19_65_pct', 'WomanPopulationPtge'), ('Age_under19_Ptge', 'Age_over65_pct'), ('DifComAutonPtge', 'SUPERFICIE'), ('ForeignersPtge', 'Unemploy25_40_Ptge'), ('ForeignersPtge', 'DifComAutonPtge'), ('Age_over65_pct', 'SameComAutonDiffProvPtge'), ('DifComAutonPtge', 'UnemployMore40_Ptge'), ('UnemployMore40_Ptge', 'AgricultureUnemploymentPtge'))]]

8.3. Validación cruzada

Tras los métodos de selección, nos quedamos con 5 modelos los cuales fueron comparados mediante la validación cruzada. Tras aplicar la validación cruzada, se obtuvieron los siguientes diagramas de caja y bigotes en base al valor de R2



Para obtener una comparación más clara, se obtuvieron los valores numéricos de la media de R2, desviación estándar de R2 y el número de parámetros para los 5 modelos, dando lugar a la siguiente tabla

	Model	Mean R2	Std R2	Num Parameters
Modelo				
1.0	StepwiseBIC	0.4119	0.0162	63
2.0	Random Selection 1	0.4220	0.0185	63
3.0	Random Selection 2	0.4271	0.0196	68
4.0	Random Selection 3	0.4256	0.0159	69
5.0	StepwiseBIC with interactions	0.4301	0.0168	71

Por el principio de parsimonia, podríamos pensar en elegir el modelo 1 o el 2. Ambos son los que menos parámetros tienen y mantienen un valor de R2 muy similar. Sin embargo, el modelo 2 incluye interacciones, lo que lo hace más complejo. No merece la pena sacrificar la simpleza del modelo 1 a cambio de un incremento en R2 tan pequeño. Además, las interacciones introducen bastante colinealidad, reduciendo la estabilidad de los parámetros del modelo.

Aun así, cabe mencionar que todos los modelos de regresión cuentan con un R2 bastante normal tirando a bajo además de bastante colinealidad. Esto se puede deber a las relaciones descubiertas entre porcentajes de población en rangos de edad y las relaciones entre número de empresas por sector. Al final, muchas variables introducidas en el estudio son porcentajes, los cuales para dar el 100% dependen unos de otros.

En el caso de los porcentajes de población en rangos de edad, parece ser que no es la mejor forma de medir la edad de un grupo de la población. Quizá una variable numérica discreta sería la mejor opción, ya que realmente hay un orden en los datos y representamos todo en una misma columna.

8.4. Análisis del modelo ganador

A continuación, podemos observar el resumen del modelo ganador:

OLS Regression Results			
Dep. Variable:	y	R-squared:	0.424
Model:	OLS	Adj. R-squared:	0.419
Method:	Least Squares	F-statistic:	76.46
Date:	Sun, 09 Feb 2025	Prob (F-statistic):	0.00
Time:	11:22:08	Log-Likelihood:	-20573.
No. Observations:	6493	AIC:	4.127e+04
Df Residuals:	6430	BIC:	4.170e+04
Df Model:	62		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	42.9831	1.796	23.938	0.000	39.463	46.503
SameComAutonPtge	-0.0841	0.008	-10.498	0.000	-0.100	-0.068
Explotaciones	0.0030	0.000	7.559	0.000	0.002	0.004
ConstructionUnemploymentPtge	0.0384	0.007	5.542	0.000	0.025	0.052
ServicesUnemploymentPtge	0.0175	0.003	5.282	0.000	0.011	0.024
IndustryUnemploymentPtge	0.0320	0.008	4.109	0.000	0.017	0.047
Age_19_65_pct	-0.0711	0.013	-5.272	0.000	-0.098	-0.045
Age_under19_Ptge	0.0847	0.018	4.636	0.000	0.049	0.120
WomanPopulationPtge	-0.0770	0.022	-3.534	0.000	-0.120	-0.034
SameComAutonDiffProvPtge	-0.0616	0.020	-3.019	0.003	-0.102	-0.022
CodigoProvincia_10	-3.6394	1.010	-3.605	0.000	-5.619	-1.660
CodigoProvincia_11	3.6730	1.319	2.784	0.005	1.087	6.259
CodigoProvincia_12	-9.3950	1.056	-8.893	0.000	-11.466	-7.324
CodigoProvincia_13	-6.1543	1.109	-5.547	0.000	-8.329	-3.979
CodigoProvincia_14	-5.4831	1.189	-4.611	0.000	-7.814	-3.152
CodigoProvincia_15	1.2839	1.158	1.109	0.267	-0.985	3.553
CodigoProvincia_16	-7.8432	0.990	-7.921	0.000	-9.784	-5.902
CodigoProvincia_17	4.2757	0.987	4.330	0.000	2.340	6.211
CodigoProvincia_18	-2.0930	1.032	-2.028	0.043	-4.116	-0.070
CodigoProvincia_19	-9.8604	0.985	-10.010	0.000	-11.792	-7.929
CodigoProvincia_2	-7.7893	1.165	-6.684	0.000	-10.074	-5.505
CodigoProvincia_20	5.6109	1.125	4.986	0.000	3.405	7.817
CodigoProvincia_21	2.7156	1.156	2.349	0.019	0.450	4.982
CodigoProvincia_22	-3.4914	1.003	-3.480	0.001	-5.458	-1.525
CodigoProvincia_23	-6.7673	1.166	-5.806	0.000	-9.052	-4.482
CodigoProvincia_24	-1.2524	1.013	-1.237	0.216	-3.238	0.733
CodigoProvincia_25	7.2322	0.987	7.328	0.000	5.298	9.167
CodigoProvincia_26	-10.6650	1.035	-10.302	0.000	-12.694	-8.636
CodigoProvincia_27	0.9094	1.234	0.737	0.461	-1.510	3.329
CodigoProvincia_28	-6.9692	1.031	-6.761	0.000	-8.990	-4.948
CodigoProvincia_29	-1.1165	1.130	-0.988	0.323	-3.332	1.099
CodigoProvincia_3	-9.8151	1.043	-9.409	0.000	-11.860	-7.770
CodigoProvincia_30	-6.3873	1.384	-4.614	0.000	-9.101	-3.673
CodigoProvincia_31	-0.1780	0.983	-0.181	0.856	-2.104	1.748
CodigoProvincia_32	-1.2123	1.132	-1.071	0.284	-3.431	1.006
CodigoProvincia_33	2.9117	1.175	2.478	0.013	0.608	5.215
CodigoProvincia_34	-6.1966	1.021	-6.069	0.000	-8.198	-4.195
CodigoProvincia_35	4.1696	1.438	2.899	0.004	1.350	6.989
CodigoProvincia_36	-0.5572	1.212	-0.460	0.646	-2.933	1.819
CodigoProvincia_37	-3.9843	0.970	-4.109	0.000	-5.885	-2.083
CodigoProvincia_38	2.5689	1.262	2.036	0.042	0.096	5.042
CodigoProvincia_39	-3.6889	1.092	-3.379	0.001	-5.829	-1.549
CodigoProvincia_4	-4.5176	1.103	-4.096	0.000	-6.680	-2.355
CodigoProvincia_40	-8.1657	1.001	-8.155	0.000	-10.129	-6.203
CodigoProvincia_41	-0.9868	1.122	-0.879	0.379	-3.187	1.213
CodigoProvincia_42	-3.7549	1.033	-3.635	0.000	-5.780	-1.730
CodigoProvincia_43	2.4216	1.001	2.419	0.016	0.459	4.384
CodigoProvincia_44	-5.8574	0.987	-5.935	0.000	-7.792	-3.923
CodigoProvincia_45	-7.3573	1.005	-7.324	0.000	-9.327	-5.388
CodigoProvincia_46	-9.5786	0.988	-9.691	0.000	-11.516	-7.641
CodigoProvincia_47	-7.1662	0.993	-7.215	0.000	-9.113	-5.219
CodigoProvincia_48	-1.2488	1.081	-1.155	0.248	-3.368	0.871
CodigoProvincia_49	-2.2751	0.997	-2.281	0.023	-4.230	-0.320
CodigoProvincia_5	-7.2558	0.994	-7.300	0.000	-9.204	-5.307
CodigoProvincia_50	-4.0117	0.975	-4.116	0.000	-5.922	-2.101
CodigoProvincia_6	-3.0665	1.045	-2.933	0.003	-5.116	-1.017
CodigoProvincia_7	1.1661	1.222	0.954	0.340	-1.229	3.562
CodigoProvincia_8	2.2596	0.971	2.328	0.020	0.357	4.163
CodigoProvincia_9	-4.8046	0.960	-5.003	0.000	-6.687	-2.922
ActividadPpal_Construccion	-3.2455	1.759	-1.845	0.065	-6.694	0.203
ActividadPpal_Industria	0.6074	1.760	0.345	0.730	-2.842	4.057
ActividadPpal_Otro	-1.9884	0.229	-8.670	0.000	-2.438	-1.539
ActividadPpal_Servicios	-0.6910	0.316	-2.185	0.029	-1.311	-0.071
Omnibus:	266.692	Durbin-Watson:	2.035			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	681.231			
Skew:	0.205	Prob(JB):	1.18e-148			
Kurtosis:	4.533	Cond. No.	2.31e+04			

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.31e+04. This might indicate that there are strong multicollinearity or other numerical problems.

Además, podemos observar su número de parámetros y los valores de media y desviación estándar de R2

Model	Mean R2	Std R2	Num Parameters
StepwiseBIC	0.4119	0.0162	63

8.5. Interpretación de los coeficientes

CodigoProvincia_11 (3.6730)

- Este coeficiente indica que, si un municipio pertenece a la provincia con el código 11 (Cádiz), la abstención aumenta en 3.6730 puntos porcentuales en comparación con los municipios de la provincia de referencia (la provincia omitida en el modelo: Álava).

WomanPopulationPtge (-0.0770)

- Por cada punto porcentual que aumenta la población femenina en un municipio, la abstención disminuye en 0.0610 puntos porcentuales.
- El coeficiente negativo indica una relación inversa: a mayor proporción de mujeres, menor abstención.

Cabe destacar que estos efectos se interpretan manteniendo todas las demás variables del modelo constantes

9. Regresión logística

Para la regresión logística, se siguió el mismo procedimiento que en la regresión lineal: dividir el conjunto de datos en entrenamiento y prueba antes de aplicar los métodos de selección de variables.

Sin embargo, debido a las limitaciones de tiempo y recursos computacionales, decidí no incluir interacciones en esta etapa. Aunque esto podría afectar la calificación, consideré que era una decisión justificada. En la regresión lineal, el principio de parsimonia mostró que el modelo más simple, sin interacciones, fue el mejor pese a una ligera reducción en R². Dado que trabajamos con el mismo conjunto de datos, asumí que en la regresión logística los beneficios de incluir interacciones no compensarían la pérdida de eficiencia y sencillez del modelo.

Por estas razones, opté por un modelo sin interacciones, priorizando la simplicidad y la viabilidad computacional.

9.1. Selección clásica de variables

Igual que en regresión lineal, se ejecutaron varios métodos de selección (stepwise, backward y forward) tanto para la métrica AIC como BIC). En este caso, la diferencia radica en la función utilizada: `glm_stepwise` (por ejemplo) que es distinta de `lm_stepwise`.

```
modeloStepAIC = glm_stepwise(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloBackAIC = glm_backward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloFrontAIC = glm_forward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloStepBIC = glm_stepwise(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloBackBIC = glm_backward(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloFrontBIC = glm_forward(y_train, x_train, var_cont, var_categ, [], 'BIC')
```

Tras el tiempo de ejecución, se obtuvieron los siguientes resultados:

	Pseudo-R ² Train	Pseudo-R ² Test	Number of Parameters
Stepwise AIC	0.2940	0.2973	77.0
Backward AIC	0.2947	0.2775	77.0
Forward AIC	0.2997	0.2814	90.0
Stepwise BIC	0.2940	0.2973	77.0
Backward BIC	0.2947	0.2775	77.0
Forward BIC	0.2997	0.2814	90.0

De todos estos modelos, observamos un mejor PseudoR2 y un menor número de parámetros en stepwiseAIC y y stepwiseBIC. Ambos casos obtienen los mismos resultados. Se decidió que el stepwiseBIC sería el modelo ganador de esta sección.

9.2. Selección aleatoria de variables

Para la selección aleatoria de variables, se utilizó un código muy similar al de la regresión lineal. Simplemente adaptamos el método stepwise a la regresión logística.

```
## Selección aleatoria (se coge la submuestra de los datos de entrenamiento)

# Inicializar un diccionario para almacenar las fórmulas y variables seleccionadas.
variables_seleccionadas = {
    'Formula': [],
    'Variables': [],
    'Interaccion': []
}

# Realizar 20 iteraciones de selección aleatoria. (en clase no se puede correr con 20)
for x in range(20):
    print('----- iter: ' + str(x))

    # Dividir los datos de entrenamiento en conjuntos de entrenamiento y prueba.
    x_train2, x_test2, y_train2, y_test2 = train_test_split(x_train, y_train,
                                                            test_size = 0.3, random_state = 1234567 + x)

    # Realizar la selección stepwise utilizando el criterio BIC en la submuestra.
    #modelo = glm_stepwise(y_train2.astype(int), x_train2, var_cont, var_categ, interacciones_varcont, 'BIC')
    modelo = glm_stepwise(y_train2.astype(int), x_train2, var_cont, var_categ, [], 'BIC')

    # Almacenar las variables seleccionadas y la fórmula correspondiente.
    variables_seleccionadas['Variables'].append(modelo['Variables'])
    variables_seleccionadas['Formula'].append(sorted(modelo['X'].columns))
# Unir las variables en las fórmulas seleccionadas en una sola cadena.
variables_seleccionadas['Formula'] = list(map(lambda x: '+'.join(x), variables_seleccionadas['Formula']))

# Calcular la frecuencia de cada fórmula y ordenarlas por frecuencia.
frecuencias = Counter(variables_seleccionadas['Formula'])
frec_ordenada = pd.DataFrame(list(frecuencias.items()), columns = ['Formula', 'Frecuencia'])
frec_ordenada = frec_ordenada.sort_values('Frecuencia', ascending = False).reset_index()

# Identificar las tres fórmulas más frecuentes y las variables correspondientes.
var_1 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][0])]
var_2 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][1])]
var_3 = variables_seleccionadas['Variables'][variables_seleccionadas['Formula'].index(
    frec_ordenada['Formula'][2])]
```

Como resultado, obtuvimos las 3 fórmulas más frecuentes:

- Variables de la fórmula 1: {'cont': ['SameComAutonPtge', 'Age_over65_pct', 'SUPERFICIE', 'Explotaciones'], 'categ': ['CodigoProvincia', 'ActividadPpal', 'Densidad', 'CCAA'], 'inter': []}
- Variables de la fórmula 2: {'cont': ['SameComAutonPtge', 'Age_over65_pct', 'Explotaciones'], 'categ': ['CodigoProvincia', 'ActividadPpal', 'Densidad', 'CCAA'], 'inter': []}

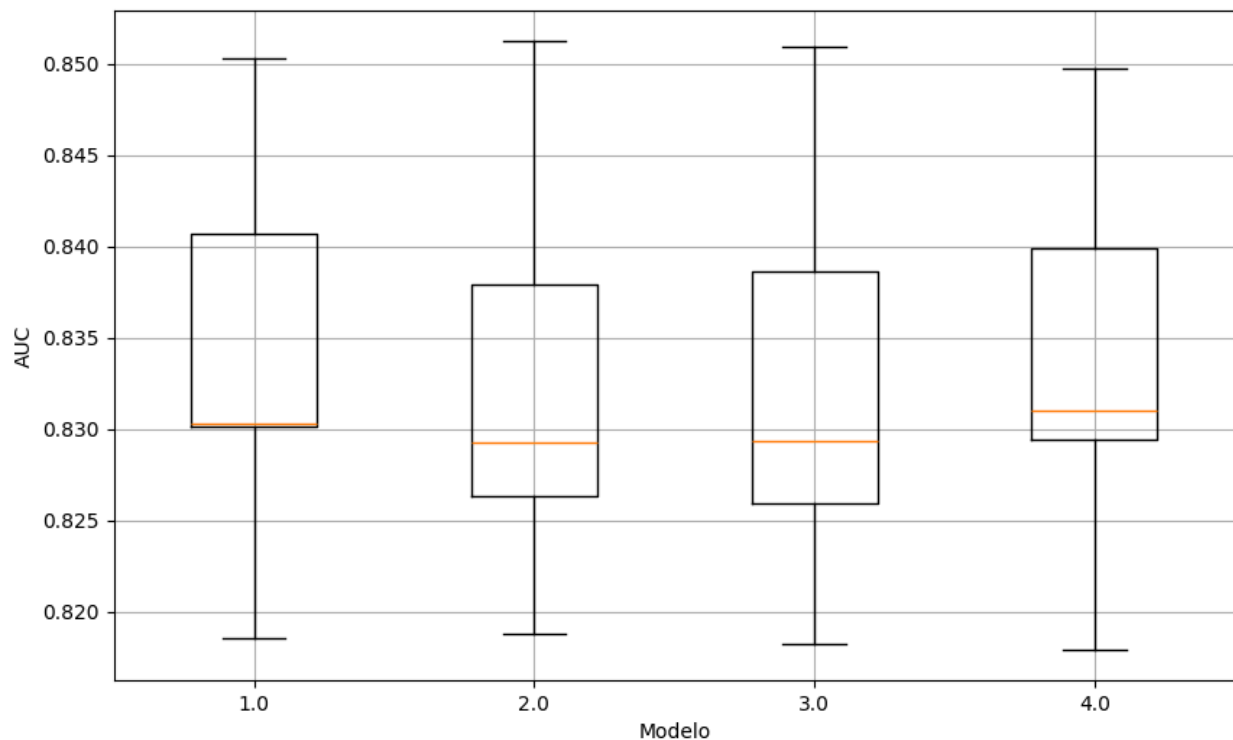
- Variables de la fórmula 3: {'cont': ['SameComAutonPtge', 'Age_19_65_pct', 'Explotaciones', 'WomanPopulationPtge', 'PobChange_pct'], 'categ': ['CodigoProvincia', 'ActividadPpal', 'CCAA', 'Densidad'], 'inter': []}

Entonces, en este punto contamos con 4 posibles candidatos a modelo ganador. A continuación, con la validación cruzada, obtendremos el elegido.

9.3. Validación cruzada

Para la validación cruzada, utilizamos el mismo procedimiento que en el caso de la regresión lineal. Sin embargo, en este caso no comparamos el valor de R2. sino el área bajo la curva ROC.

En primer lugar, vemos a continuación el gráfico de caja y bigotes que compara el valor de la AUC para los 4 modelos. Observamos la media y la varianza de este valor comparada.



Para obtener un resultado más exacto, contamos con este trozo de código que nos permite obtener en variables almacenadas estos valores de media y desviación estándar de la AUC.

```
# Calcular la media de las métricas R-squared por modelo
media_AUC2= results2.groupby('Modelo')['AUC'].mean()
# Calcular la desviación estándar de las métricas R-squared por modelo
std_AUC2 = results2.groupby('Modelo')['AUC'].std()
# Contar el número de parámetros en cada modelo
num_params2 = [len(modeloStepBIC['Modelo'].coef_[0]),
               len(frec_ordenada['Formula'][0].split('+')),
               len(frec_ordenada['Formula'][1].split('+')),
               len(frec_ordenada['Formula'][2].split('+'))]
```

Además, es fácilmente visualizable en una tabla con el siguiente código

```
# Create DataFrame with metrics
metrics_df = pd.DataFrame({
    'Model': ['StepwiseBIC', 'Random Selection 1', 'Random Selection 2', 'Random Selection 3'],
    'Mean AUC': media_AUC2,
    'Std AUC': std_AUC2,
    'Num Parameters': [len(modeloStepBIC['Modelo'].coef_[0]),
                      len(frec_ordenada['Formula'][0].split('+')),
                      len(frec_ordenada['Formula'][1].split('+')),
                      len(frec_ordenada['Formula'][2].split('+'))]
})

# Format floating point numbers
metrics_df['Mean AUC'] = metrics_df['Mean AUC'].round(4)
metrics_df['Std AUC'] = metrics_df['Std AUC'].round(4)

print(metrics_df)
```

El resultado es el siguiente

	Model	Mean AUC	Std AUC	Num Parameters
Modelo				
1.0	StepwiseBIC	0.8340	0.0108	77
2.0	Random Selection 1	0.8327	0.0112	75
3.0	Random Selection 2	0.8326	0.0113	74
4.0	Random Selection 3	0.8336	0.0107	76

Siguiendo el principio de parsimonia, el modelo ganador es el 3. Todos los modelos son muy parecidos, pero este modelo cuenta con el número de parámetros más bajo

9.4. Selección del punto de corte

Ahora que tenemos un único modelo, vamos a buscar el mejor punto de corte mediante el siguiente código:

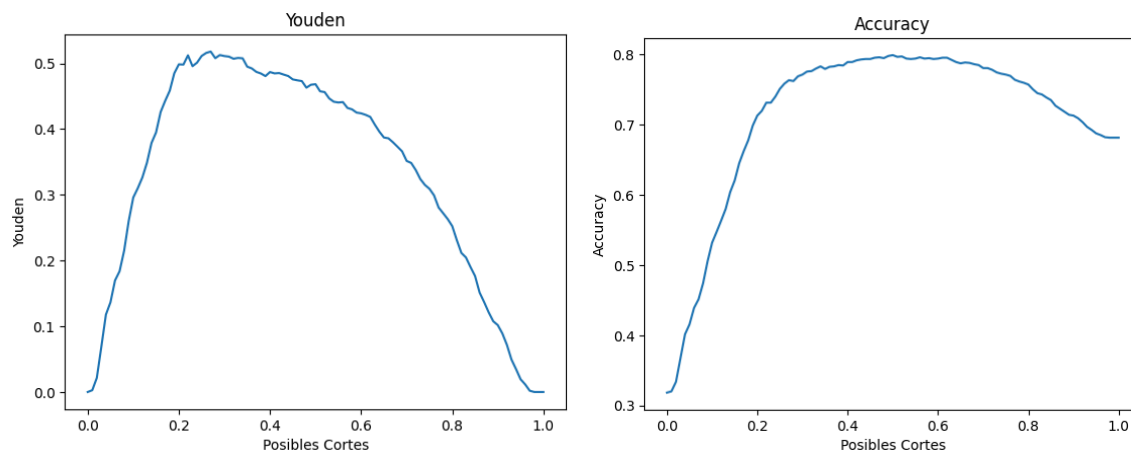
```
# Generamos una rejilla de puntos de corte
posiblesCortes = np.arange(0, 1.01, 0.01).tolist() # Generamos puntos de corte de 0 a 1 con intervalo de 0.01
rejilla = pd.DataFrame({
    'PtoCorte': [],
    'Accuracy': [],
    'Sensitivity': [],
    'Specificity': [],
    'PosPredValue': [],
    'NegPredValue': []
}) # Creamos un DataFrame para almacenar las métricas para cada punto de corte

for pto_corte in posiblesCortes: # Iteramos sobre los puntos de corte
    rejilla = pd.concat(
        [rejilla, sensEspCorte(modeloGanador['Modelo'], x_test, y_test, pto_corte, var_contGanador, var_categGanador)],
        axis=0
    ) # Calculamos las métricas para el punto de corte actual y lo agregamos al DataFrame

rejilla['Youden'] = rejilla['Sensitivity'] + rejilla['Specificity'] - 1 # Calculamos el índice de Youden
rejilla.index = list(range(len(rejilla))) # Reindexamos el DataFrame para que los índices sean consecutivos
```

El objetivo es encontrar aquel punto de corte representado por la curva ROC que mejores prestaciones de sensibilidad y especificidad nos de para nuestro caso. Para ello, se han calculado el índice de Youden y la precisión para todos los puntos de corte posibles (de 0 a 1 con intervalo de 0,01).

Como resultado, tenemos las siguientes gráficas



Para ambas métricas, encontramos el punto de corte que las maximiza:

```
# Encuentra el punto de corte que maximiza el índice de Youden
pto_corte_youden = rejilla['PtoCorte'][rejilla['Youden'].idxmax()]

# Encuentra el punto de corte que maximiza la precisión (Accuracy)
pto_corte_accuracy = rejilla['PtoCorte'][rejilla['Accuracy'].idxmax()]
print(f"Punto de corte que maximiza el índice de Youden: {pto_corte_youden}")
print(f"Punto de corte que maximiza la precisión (Accuracy): {pto_corte_accuracy}")
```

Como resultado obtenemos que para el índice de Youden, el punto de corte ideal es 0.27 mientras que para la precisión es 0.5. Comparando ambos puntos de corte, observamos lo siguiente. El índice de Youden busca un equilibrio entre sensibilidad y especificidad, mientras que vemos como la precisión potencia mucho más la especificidad.

En este caso, la sensibilidad del punto de corte 0.5 es demasiado baja: casi la mitad de las predicciones positivas ($y = 1$) son falsas. Por lo tanto, tenemos que elegir algo más equilibrado, y el punto de corte seleccionado es el 0.27.

	PtoCorte	Accuracy	Sensitivity	Specificity	PosPredValue	NegPredValue
0	0.27	0.763547	0.746615	0.771454	0.604069	0.867005

	PtoCorte	Accuracy	Sensitivity	Specificity	PosPredValue	NegPredValue
0	0.5	0.799261	0.555126	0.913279	0.749347	0.814666

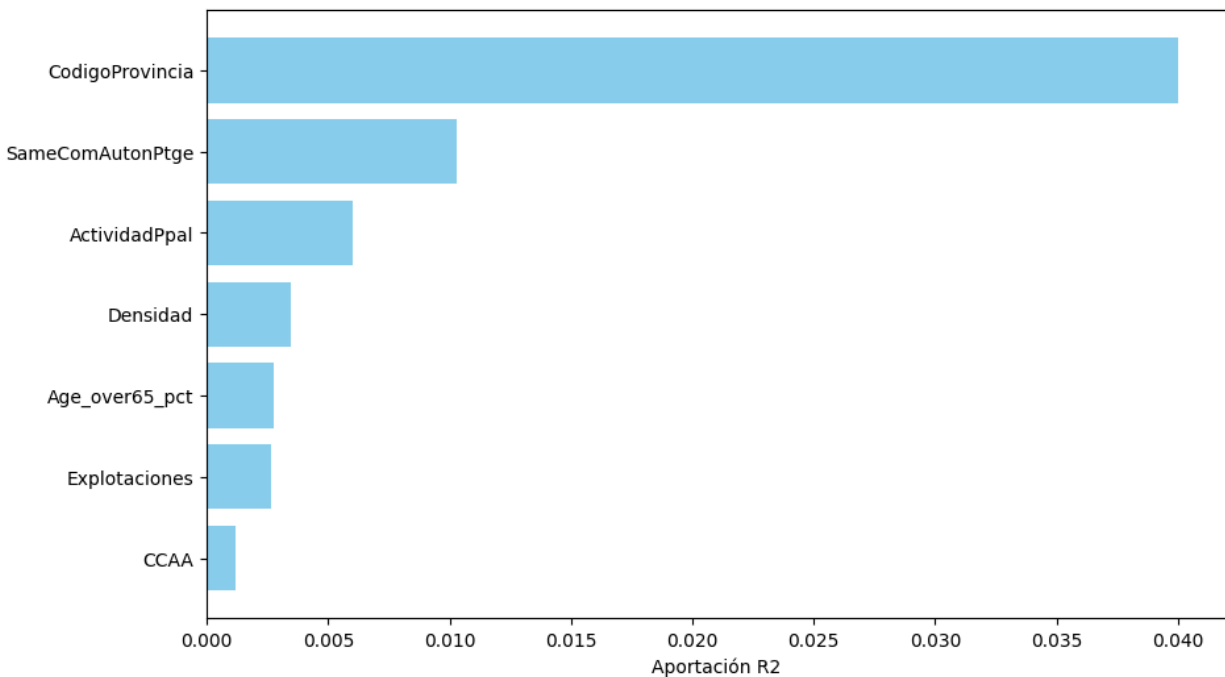
9.5. Análisis del modelo ganador e interpretación de los coeficientes

En primer lugar, tenemos aquí el resumen del modelo

```
print(summary_glm(modeloGanador['Modelo'], y_train, modeloGanador['X']))
```

{'Contrastes':						
	Variable	Estimate	Std. Error	z value	p value	signif
0	(Intercept)	2.263998	NaN	NaN	nan	
1	SameComAutonPtge	-0.031890	7.951030e-03	-4.010748e+00	0.000061	***
2	Age_over65_pct	0.019690	4.149020e-03	4.745591e+00	0.000002	***
3	Explotaciones	0.000728	1.734879e-04	4.197988e+00	0.000027	***
4	CodigoProvincia_10	-0.364156	1.544727e+07	-2.357410e-08	1.000000	
..	...	...	...	...	...	...
70	CCAA_Madrid	-0.771688	NaN	NaN	nan	
71	CCAA_Murcia	-0.501129	NaN	NaN	nan	
72	CCAA_Navarra	0.338644	4.737199e+07	7.148613e-09	1.000000	
73	CCAA_PaísVasco	0.552695	NaN	NaN	nan	
74	CCAA_Rioja	-1.357415	1.424683e+08	-9.527839e-09	1.000000	
[75 rows x 6 columns], 'BondadAjuste':			LLK	AIC	BIC	
0	-2834.233174	5672.466348	5686.023308}			

A continuación, la importancia de cada variable al valor de pseudoR2. Podemos ver como la provincia sigue siendo igual de importante para la predicción tanto en la regresión logística como en la regresión lineal.



Ahora, la interpretación de los coeficientes de dos variables.

Interpretación del coeficiente:

- Variable: Age_over65_pct
- Coeficiente: 0.019689554115921024

Un aumento de una unidad en Age_over65_pct (porcentaje de población mayor de 65 años) se asocia con un aumento de 0.0197 en el logaritmo de las probabilidades de la variable dependiente

Si calculamos el odds ratio, esto significa que por cada aumento de una unidad en Age_over65_pct, las probabilidades de la variable dependiente aumentan en un factor de aproximadamente 1.0199. Es decir, que aumenta en esas unidades que la abstención sea alta

Interpretación del coeficiente:

- Variable: CCAA_Galicia
- Coeficiente: 0.47132112067473897

El coeficiente 0.4713 indica que si un municipio pertenece a Galicia (en comparación con la categoría de referencia de la variable CCAA, que es Andalucía), el logaritmo de las probabilidades de la variable dependiente aumenta en 0.4713, manteniendo constantes las demás variables del modelo. Esto significa que pertenecer a Galicia está asociado con un aumento en las probabilidades de la variable dependiente en un factor de aproximadamente 1.6019 en comparación con pertenecer a Andalucía.

Por último, vamos a ver la tabla comparativa entre las métricas pseudoR2 y AUROC de los conjuntos de entrenamiento y test para el modelo ganador:

	Métrica	Entrenamiento	Test
0	Pseudo R2	0.294397	0.284214
1	ROC AUC	0.841327	0.839217

Observamos que los valores son muy similares. No hay presencia de sobreajuste y por lo tanto el modelo ganador es robusto.